7.1 MIPS CPU概述

同濟大學
TONGJI UNIVERSITY

目录 | CONTENT

1 7.1.1 概述

2 7.1.2 基本架构及编程模型

3 7.1.5 Mars汇编器

7.1.1 概述

- MIPS的全名为“Microcomputer without interlocked pipeline stages”，是最早最成功的RISC（Reduced Instruction Set Computer）处理器之一。
- John Hennessy教授和他的学生们探寻了精简指令集（RISC）体系结构概念，并成立了MIPS计算机系统公司，该概念基于如下理论：使用相对简单的指令，结合优秀的编译器以及采用流水线执行指令的硬件，就可以用更少的晶元面积生产更快的处理器。
- MIPS在RISC处理器方面占有重要地位，是最早的RISC微处理器以及较早的超标量与64位微处理器。
- MIPS微处理器经常能在处理器的每个技术发展阶段保持速度最快的同时保持设计的简洁。
- 在设计理念上MIPS强调软硬件协同提高性能，同时简化硬件设计。

7.1.2 基本架构及编程模型

本课程所讲的基本架构为MIPS32架构。

- MIPS32架构中有32个通用寄存器，虽然硬件没有强制性的指定寄存器使用规则，但在实际使用中，这些寄存器的用法都遵循一系列约定。这些约定与硬件确实无关，但如果你想使用别人的代码、编译器和操作系统，最好是遵循这些约定。通用寄存器的命名约定及用法如表7.1.1所示。
- MIPS32中没有条件码，比如在X86中常见的标志寄存器中的Z、C标志位在MIPS32中是没有的，但是MIPS32中是有状态寄存器的。
- MIPS32中不同于其它的RISC架构的地方是其有整数乘法部件，这个部件使用两个特殊的寄存器HI、LO，并且提供相应的指令 mfhi/mthi,mflo/mtlo来实现整数乘法结果HI/LO寄存器与通用寄存器之间的数据交换。

寄存器编号	记忆符	用　途
0	zero	总是返回0
1	at	汇编暂存寄存器
2~3	v0~v1	子程序返回值
4~7	a0~a3	调用子程序参数
8~15	t0~t7	暂存器
16~23	s0~s7	通用寄存器
24~25	t8~t9	暂存器
26~27	k0~k1	中断/自陷寄存器
28	gp	全局指针
29	sp	堆栈指针
30	s8/fp	通用寄存器/帧指针
31	ra	子程序返回地址

表7.1.1 寄存器命名及用法

7.1.2 基本架构及编程模型

- MIPS32中如果有FPA（浮点协处理器），将会有32个浮点寄存器，按汇编语言的约定为$f0~$f31

31不深入研究 54例

- CP0（协处理器0）受MIPS处理器控制，用于控制和设置MIPS CPU，里面包含了一些寄存器，通过对这些寄存器的不同的位的操作可以实现对处理器的设置， CP0提供了中断异常处理、内存管理（包括CACHE、TLB）、外设管理等功能。
- MIPS32中的存储器模型被划分为四个大块，如图7.1.1所示

7.1.2 基本架构及编程模型

0x0000 0000~0x7fff ffff（0~2G-1）Kuseg
must be mapped（set page table and TLB）and set cache before use

0x8000 0000~0x9fff ffff（2G~2.5G-1）Kseg0
directly mapped（no need to set page table and TLB）but need to set cache before use

0xa000 0000~0xbfff ffff（2.5G~3G-1） Kseg1
directly mapped（no need to set page table and TLB）and never use cache

0xc000，0000~0xffff ffff（3G~4G-1） Kseg2
must be mapped（set page table and TLB）and set cache before use

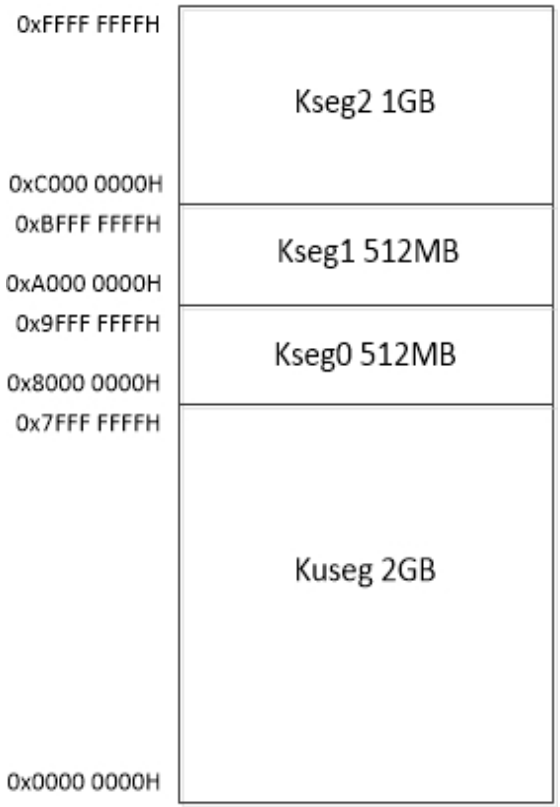


图7.1.1 存储器模型

7.1.5 Mars汇编器

MARS（the Mips Assembly and Runtime Simulator）

- MIPS 汇编程序运行模拟器，可以通过命令行和集成开发环境两种途径运行。该软件主要用于模拟真实 MIPS 处理器，运行 MIPS汇编程序。用户通过查看运行结果检验汇编程序的正确性。

- 软件运行需要 JAVA 运行环境。在 MARS 的集成开发环境中，用户可以编辑或导入 MIPS 汇编程序，编译并运行或调试程序。用户可以设置、移除断点，并向前、向后分步执行，或者在运行过程中修改寄存器和内存内容。

用户界面

- 菜单和工具栏：最常用的工具，包括新建、导入文件，汇编，执行，单步等，把鼠标箭头移到按钮上会有提示；
- 编辑窗口：编辑 MIPS 汇编代码；
- 执行窗口：汇编之后进入运行状态，执行窗口会有信息。该窗口包括三部分：代码段（显示要运行的代码，包括汇编和二进制形式以及地址）、数据段（显示用到的数据在内存中的值）和标签栏（显示跳转的标签地址）；
- 代码段：显示要运行的代码，包括汇编和二进制形式以及地址；
- 数据段：显示用到的数据在内存中的值；
- 标签栏：显示跳转的标签地址；
- 消息区域：执行反馈，包括程序结果和错误信息等；
- MIPS 寄存器：显示 MIPS 处理器 32 个通用寄存器以及 PC 的值。如图7.1.3所示。

请在这里输入标题

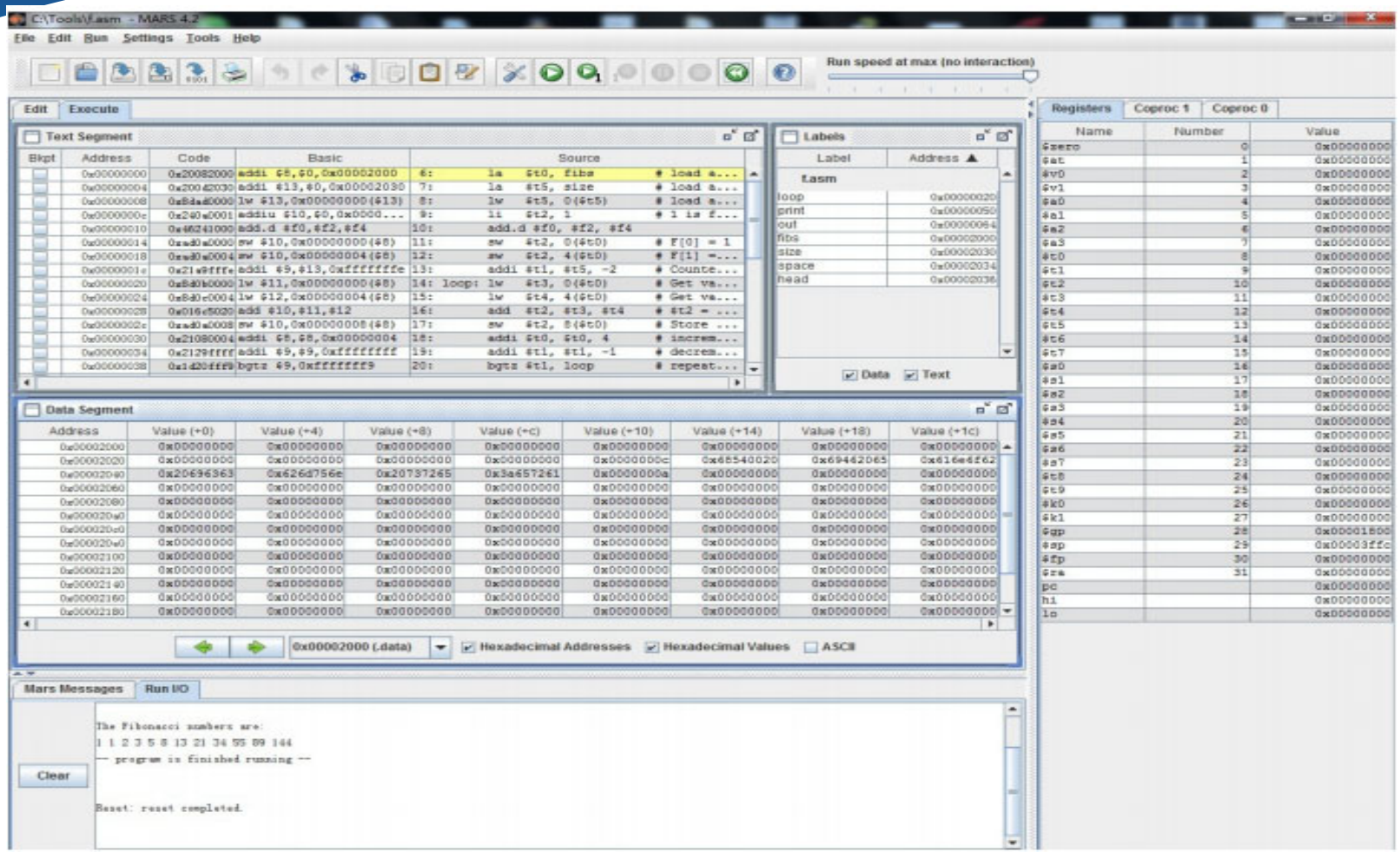
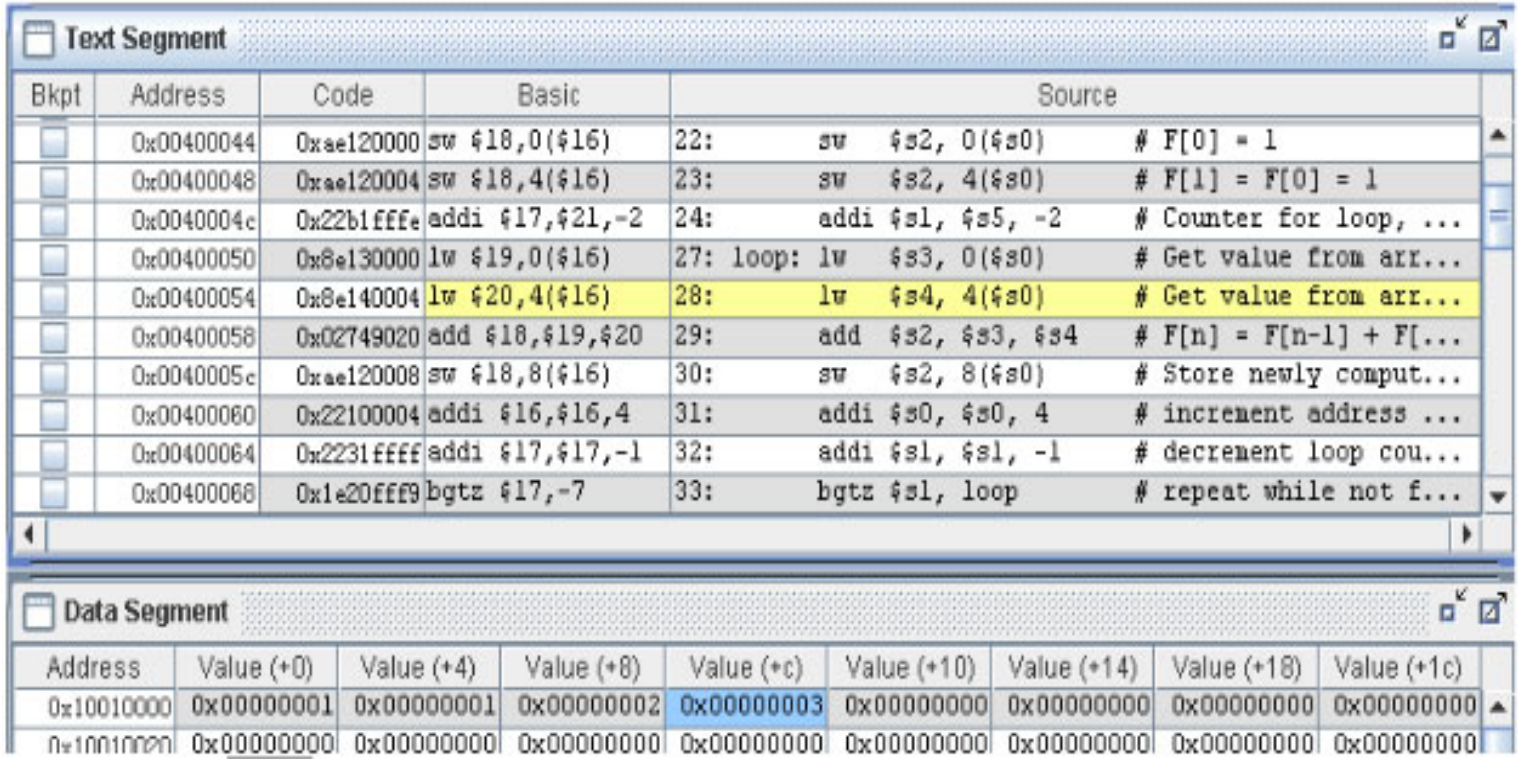


图7.1.4 MARS用户界面

基本使用

- 导入汇编程序：File->Open，找到汇编程序，打开。或者 File->New，在编辑窗口编辑汇编代码。
- 汇编：Run->Assemble，如果代码有错，在信息窗口会有提示，根据提示修改代码。否则正常进入执行窗口。
- 完整运行：Run->Go ，或者 F5
- 分步运行：Run->Step ，或者 F7
- 设置断点：在代码段最左侧的复选框中打钩设置断点。当程序运行到断点会自动停止。取消打钩就取消断点。
- 检验结果：可通过三种途径检查程序的正确性。一是看寄存器状态，查看寄存器中的值是否跟预想的一致；二是看内存值，看程序是否影响了特定的内存地址；三是程序的文本输出，在消息窗口中看，如图7.1.4所示。

请在这里输入标题



Text Segment

Bkpt	Address	Code	Basic	Source
	0x00400044	0xae120000	sw $18,0($16)	22: sw $s2, 0($s0) # F[0] = 1
	0x00400048	0xae120004	sw $18,4($16)	23: sw $s2, 4($s0) # F[1] = F[0] = 1
	0x0040004c	0x22b1fffe	addi $17,$21,-2	24: addi $s1, $s5, -2 # Counter for loop, ...
	0x00400050	0x8e130000	lw $19,0($16)	27: loop: lw $s3, 0($s0) # Get value from arr...
	0x00400054	0x8e140004	lw $20,4($16)	28: lw $s4, 4($s0) # Get value from arr...
	0x00400058	0x02749020	add $18,$19,$20	29: add $s2, $s3, $s4 # F[n] = F[n-1] + F[...
	0x0040005c	0xae120008	sw $18,8($16)	30: sw $s2, 8($s0) # Store newly comput...
	0x00400060	0x22100004	addi $16,$16,4	31: addi $s0, $s0, 4 # increment address ...
	0x00400064	0x2231ffff	addi $17,$17,-1	32: addi $s1, $s1, -1 # decrement loop cou...
	0x00400068	0x1e20fff9	bgtz $17,-7	33: bgtz $s1, loop # repeat while not f...

Data Segment

Address	Value (+0)	Value (+4)	Value (+8)	Value (+c)	Value (+10)	Value (+14)	Value (+18)	Value (+1c)
0x10010000	0x00000001	0x00000001	0x00000002	0x00000003	0x00000000	0x00000000	0x00000000	0x00000000
0x10010020	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000

图7.1.5 MARS信息窗口

谢谢聆听

Thank You